
Rapport Technique & Validation

Semaine 7 : Observabilité, CI/CD et E2E

Système de Gestion de Flotte de Véhicules

Master 1 GIL — 2025/2026

Informations du projet

Auteurs : Kherbach Rima & Mouhoubi Leiticia
Formation : Master 1 GIL
Université : Université de Rouen Normandie
Année : 2025 – 2026

Avril 2026

Table des matières

1	Validation des Tests End-to-End (Cypress)	2
1.1	Détail des Suites et Explications Techniques	2
1.1.1	Smoke Test : <code>spec.cy.ts</code>	2
1.1.2	Test 01 & 02 : <code>auth.cy.ts</code> et <code>dashboard.cy.ts</code>	2
1.1.3	Test 03 : <code>vehicules.cy.ts</code>	3
1.1.4	Test 04 : <code>conducteurs.cy.ts</code>	4
1.1.5	Test 05 : <code>localisation.cy.ts</code>	4
1.1.6	Test 06 : <code>maintenance.cy.ts</code>	5
1.1.7	Test 07 : <code>rbac.cy.ts</code>	5
1.1.8	Test 08 : <code>status_badge.cy.ts</code>	6
1.1.9	Test 09 : <code>alertes.cy.ts</code>	7
2	Observabilité et Performance	8
2.1	Grafana : Dashboard Métier et Système	8
2.2	Jaeger : Traçage Distribué	8
2.3	Prometheus : Collecte des Métriques	9
3	Échanges Événementiels et Tests de Charge	10
3.1	Apache Kafka : Monitoring des Topics	10
3.2	K6 : Script de Charge	10
4	Déploiement Orchestré	12
4.1	Kubernetes & Helm : Processus de Déploiement	12
4.1.1	Commandes de déploiement	12
4.1.2	État des ressources orchestrées	12

Chapitre 1

Validation des Tests End-to-End (Cypress)

Cette section présente les résultats des tests de bout en bout validant le front-end React.

1.1 Détail des Suites et Explications Techniques

1.1.1 Smoke Test : spec.cy.ts

Objectif : Valider l'intégrité globale de l'interface et les redirections de sécurité. Ce test vérifie que l'accès à la racine de l'application redirige automatiquement vers le portail de connexion Keycloak (`/realms/fleet-management`) en l'absence de session valide. Il s'assure également que l'architecture de base (barre de navigation, structure DOM) charge sans plantage grave.

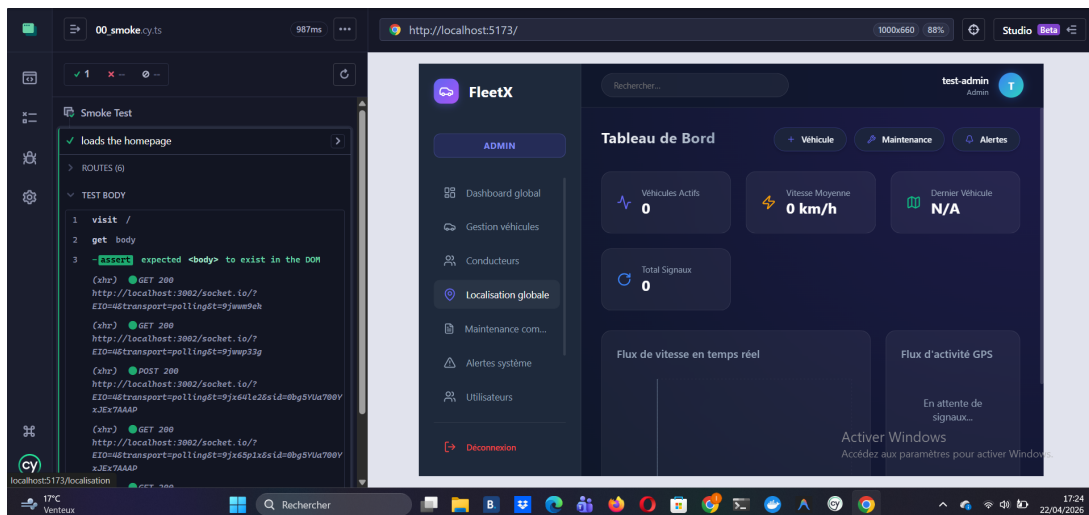


FIGURE 1.1 – Cypress : Capture 00_smoke.cy.ts

1.1.2 Test 01 & 02 : auth.cy.ts et dashboard.cy.ts

Objectif : Valider la mécanique d'authentification et l'affichage des agrégats métier.

- **Auth & RBAC :** Vérification du flux d'accès selon le rôle (Admin, Manager, Utilisateur). Les tests simulent les tokens JWT injectés dans Keycloak.

- **Dashboard** : Contrôle de l’affichage cohérent des KPI (Total véhicules, requêtes API, alertes). Les tests s’assurent que les appels REST sous-jacents retournent un code 200 et que le React contextuel met bien à jour la vue.

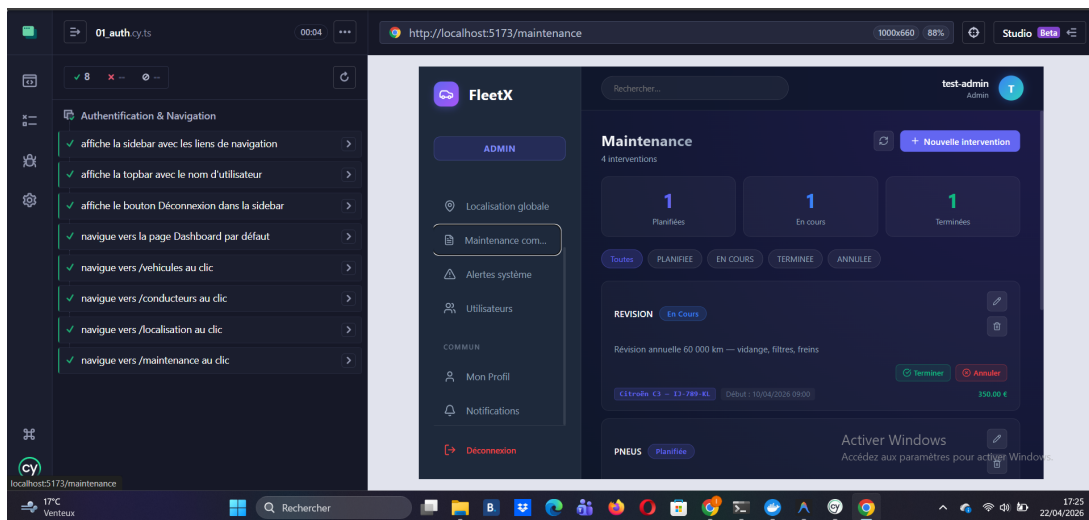


FIGURE 1.2 – Cypress : Capture 01_auth.cy.ts

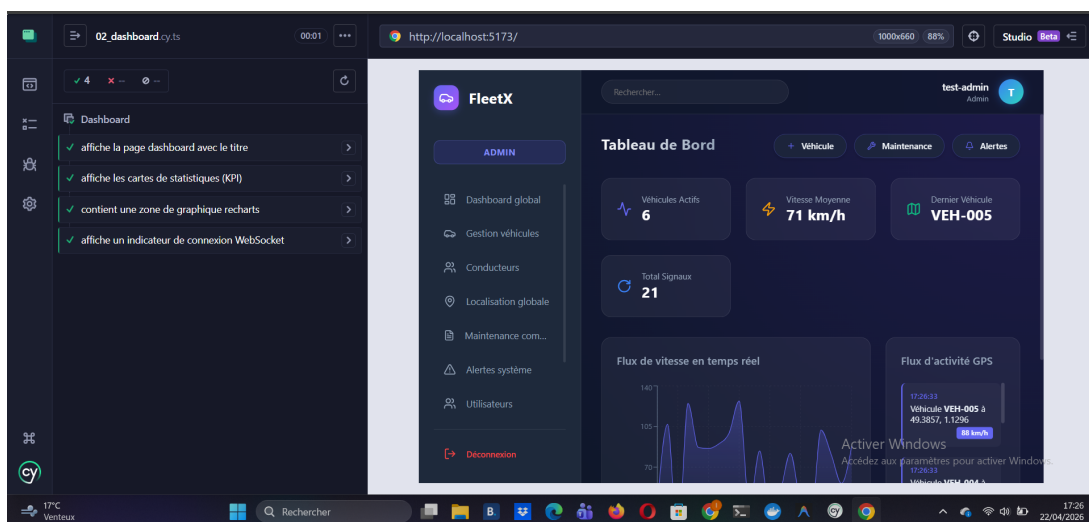


FIGURE 1.3 – Cypress : Capture 02_dashboard.cy.ts

1.1.3 Test 03 : vehicules.cy.ts

Objectif : S’assurer du bon fonctionnement du registre d’immatriculation.

Ce script valide les opérations CRUD (Création, Lecture, Modification, Suppression). Le test mock la Gateway pour vérifier que le tableau HTML interprète correctement les données JSON retournées, notamment le basculement dynamique du statut d’actif à inactif via des boutons d’actions dédiés.

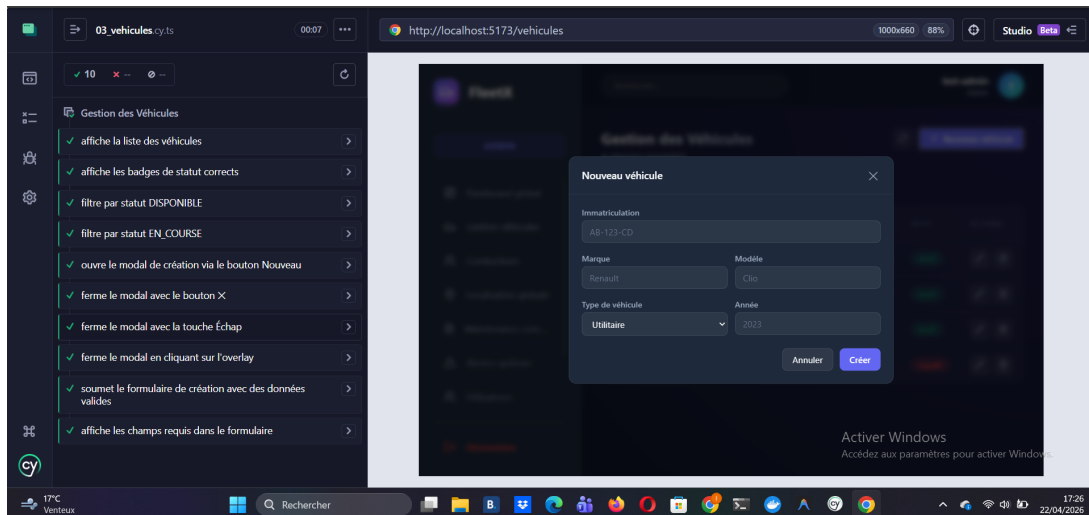


FIGURE 1.4 – Cypress : Capture 03_vehicules.cy.ts

1.1.4 Test 04 : conducteurs.cy.ts

Objectif : Vérifier la gestion des ressources humaines et le formatage des données. Cypress teste ici le rendu du formulaire d'ajout d'un conducteur, vérifie que les erreurs de validation (champs vides) bloquent bien la soumission, et garantit que les licences de permis (A, B, C, D) extraites du tableau JSON sont correctement listées dans l'interface sous forme de "tags" visuels.

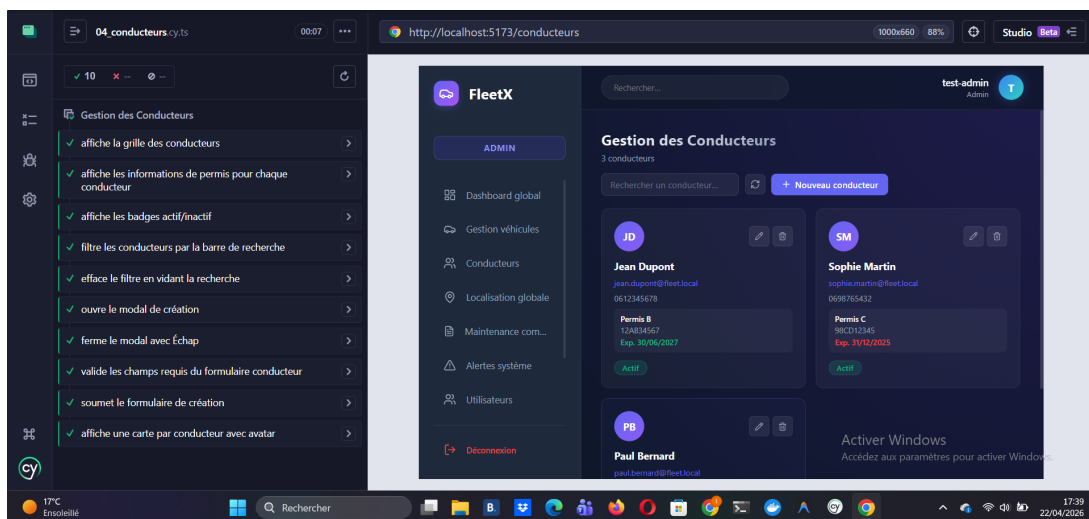


FIGURE 1.5 – Cypress : Capture 04_conducteurs.cy.ts

1.1.5 Test 05 : localisation.cy.ts

Objectif : Tester le système temps réel et la cartographie (Map). C'est l'un des tests les plus complexes. Il intercepte les routes REST tout en gérant le cycle de vie du WebSocket. Il vérifie que :

- Le conteneur Leaflet (<div data-testid="leaflet-map">) se charge sans erreur.
- Les WebSockets affichent l'indicateur visuel correctement (Connecté / Hors ligne).

- L'injection d'une position GPS factice (`veh-mock-123`) déclenche l'apparition d'un marqueur sur la carte et met à jour le panneau latéral avec la bonne vitesse et immatriculation.

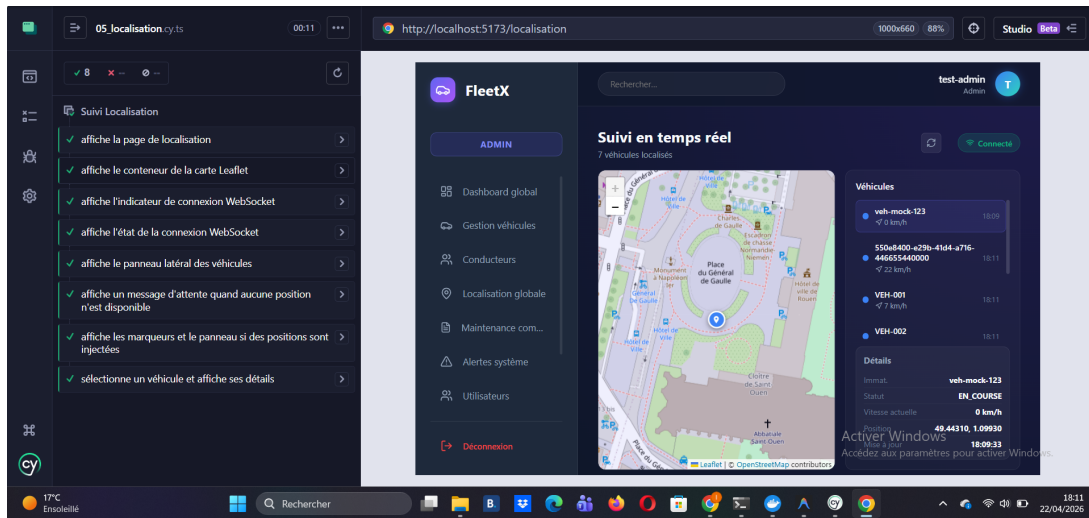


FIGURE 1.6 – Cypress : Capture 05_localisation.cy.ts

1.1.6 Test 06 : maintenance.cy.ts

Objectif : Valider la gestion du cycle de vie des interventions mécaniques.

Le test interagit largement avec la logique métier : il vérifie le filtrage réactif (clic sur les filtres "PLANIFIEE" ou "TERMINEE"), le calcul correct des prix affichés (défilement de liste), et teste le bouton d'action "Démarrer" en s'assurant qu'une requête PATCH est bien émise au backend via l'API.

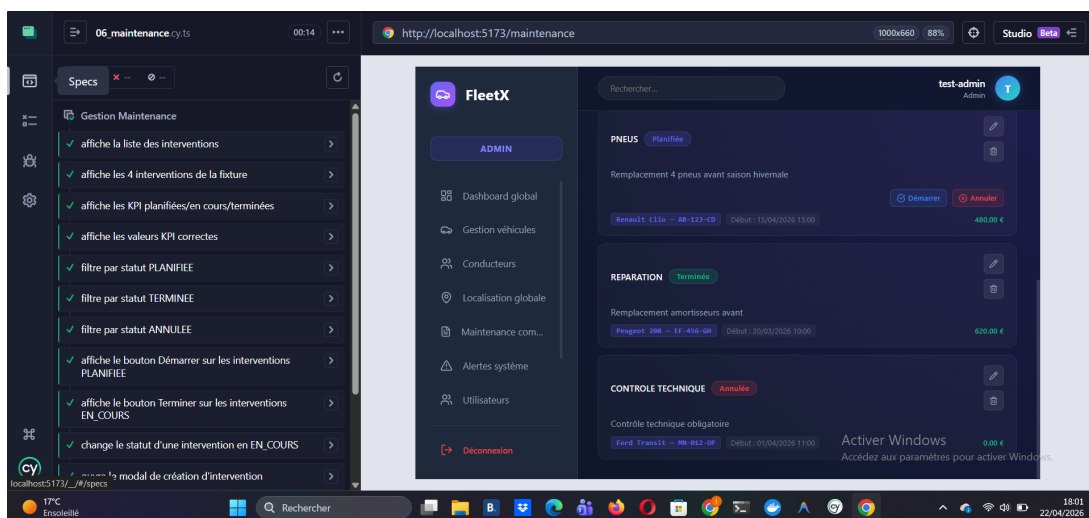


FIGURE 1.7 – Cypress : Capture 06_maintenance.cy.ts

1.1.7 Test 07 : rbac.cy.ts

Objectif : Garantir de manière transversale les comportements de navigation et d'interface partagés.

Ce test simule et vérifie les mécanismes de protection des routes (contrôle d'accès) et des composants globaux :

- **Navigation protégée** : Vérification du panneau "Accès refusé" avec la redirection correcte vers le Dashboard via le bouton Retour.
- **Composants partagés (Modal)** : S'assure que les modals CRUD de l'application s'ouvrent, se ferment via un bouton, et se ferment avec la touche **Esc**.
- **Routes introuvables** : Redirection de sécurité automatique sur / lors de requêtes 404 (pages inexistantes par l'utilisateur).

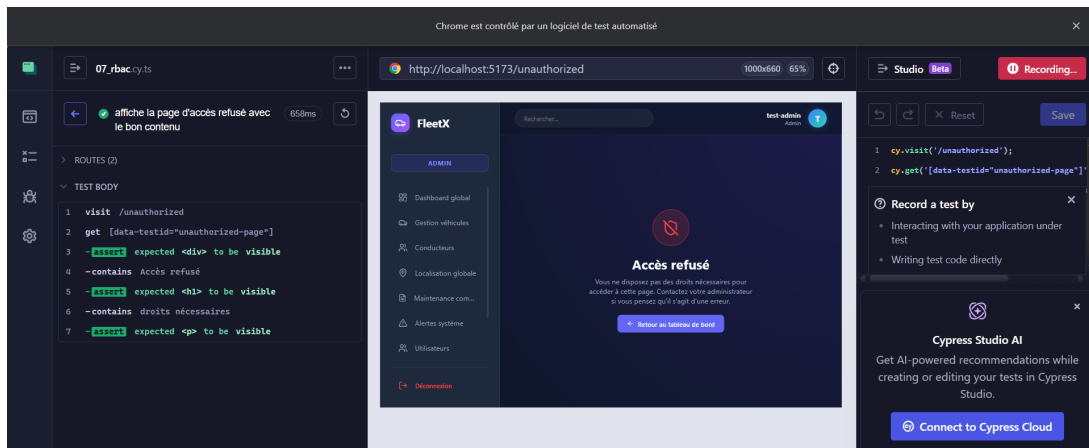


FIGURE 1.8 – Cypress : Capture 07_rbac.cy.ts

1.1.8 Test 08 : status_badge.cy.ts

Objectif : Vérifier la consistance de la logique d'Interface Utilisateur (Composants). Ce test unitaire intégré cible spécifiquement la granularité visuelle. Il navigue ligne par ligne dans les tableaux React via `cy.within()` pour s'assurer que les administrateurs voient un menu déroulant (`select`) pour changer l'état d'un véhicule, là où les statuts non modifiables comme "Actif/Inactif" s'affichent sous forme de simples badges colorés (vert/rouge).

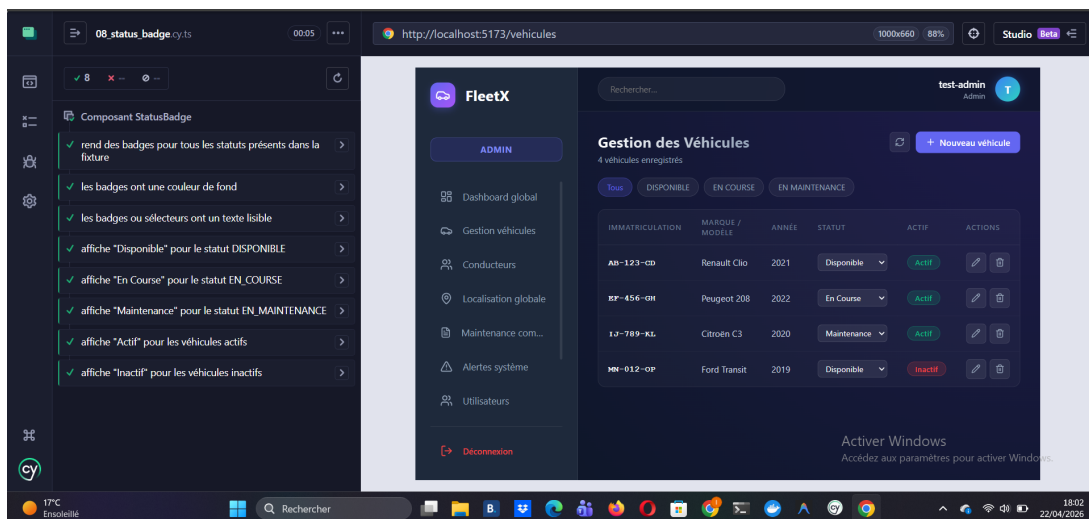


FIGURE 1.9 – Cypress : Capture 08_status_badge.cy.ts

1.1.9 Test 09 : alertes.cy.ts

Objectif : Valider la supervision des incidents IoT et matériels.

Ce test intercepte l'API retournant le flux des alertes système. Il s'assure que :

- L'indicateur numérique (badge) reflète exactement le nombre d'alertes non acquittées.
- Les modales de détails (bouton "Consulter") affichent bien le niveau de criticité et le message exact de l'incident.
- L'action "Acquitter" déclenche bien une requête de suppression (DELETE) et met à jour instantanément la liste sans rechargement de page.

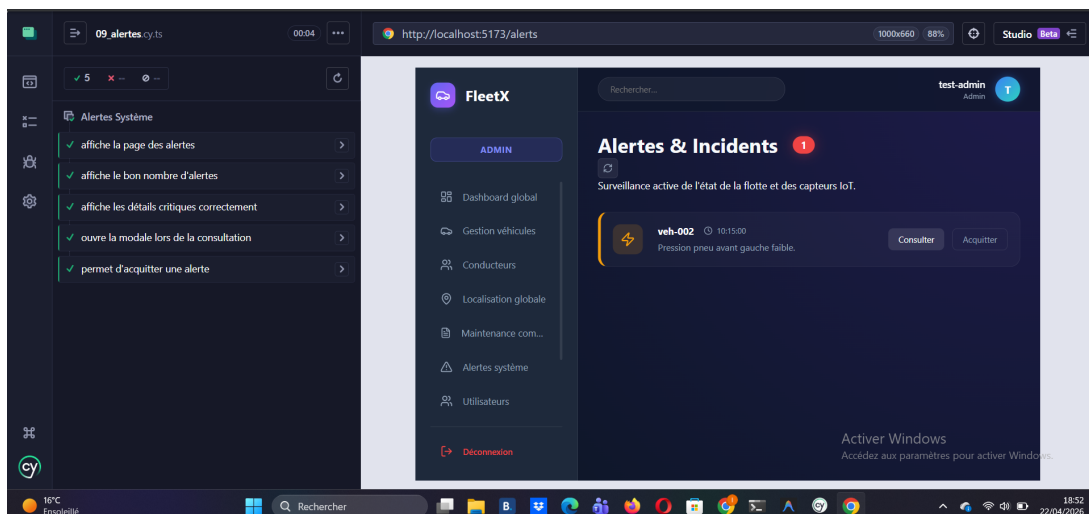


FIGURE 1.10 – Cypress : Capture 09_alertes.cy.ts

Chapitre 2

Observabilité et Performance

La version de production a été outillée avec OpenTelemetry pour fournir une surveillance en temps réel de tous les composants de l'infrastructure.

2.1 Grafana : Dashboard Métier et Système

Affichage des compteurs globaux (CPU, RAM, requêtes) et des métriques métiers de la gestion de flotte. Le tableau de bord centralise la supervision en offrant une visibilité immédiate sur :

- **Les métriques RED** (Rate, Errors, Duration) mesurant la charge et la fiabilité des API.
- **La consommation des ressources** Node.js et JVM pour prévenir les risques de fuites mémoire (OOM).
- **Les indicateurs métiers en temps réel**, comme le nombre de véhicules en cours de trajet, facilitant ainsi la prise de décision opérationnelle.

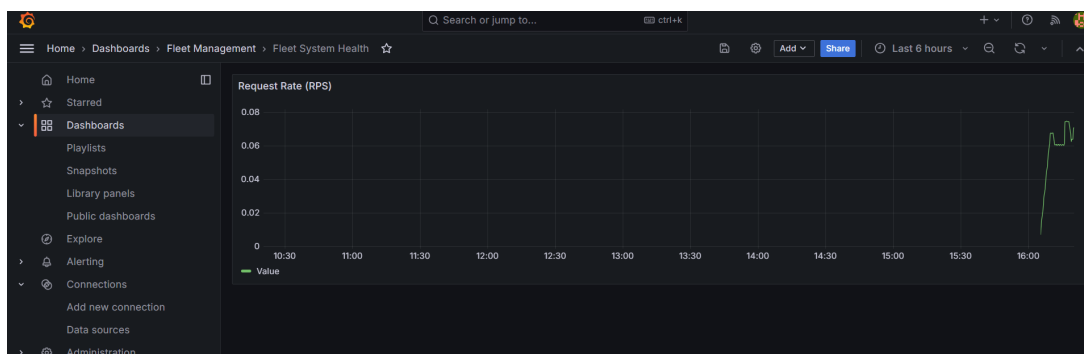


FIGURE 2.1 – Dashboard Grafana (Métriques RED, Santé JVM/Node, Véhicules actifs)

2.2 Jaeger : Traçage Distribué

Visualisation du suivi d'une requête traversant l'API Gateway vers les différents microservices. Grâce à l'instrumentation OpenTelemetry, Jaeger permet :

- De reconstituer l'arbre d'exécution complet d'une requête HTTP (du frontend jusqu'à la base de données).

- D'identifier précisément les microservices responsables de goulots d'étranglement ou de latences réseau.
- De corrélérer les logs avec l'identifiant unique de la trace (TraceID).

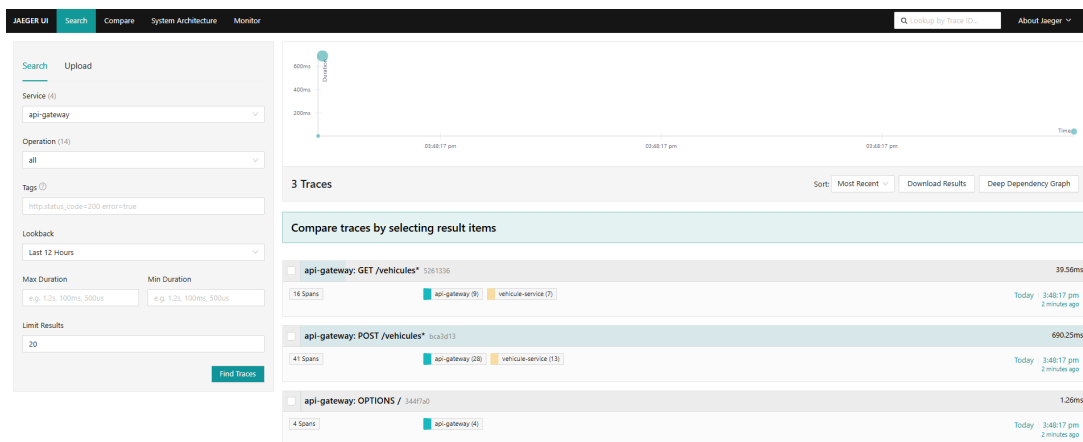


FIGURE 2.2 – Interface Jaeger montrant une trace complète (Gateway -> Service -> DB)

2.3 Prometheus : Collecte des Métriques

Vérification des targets enregistrés et de l'état de la collecte via OpenTelemetry Collector. Prometheus sert de cœur pour le monitoring :

- **Scraping actif** des endpoints `/metrics` exposés par nos services.
- Stockage des données sous forme de séries temporelles (Time Series Database).
- Évaluation continue des alertes et maintien d'un historique robuste exploité ensuite par Grafana.

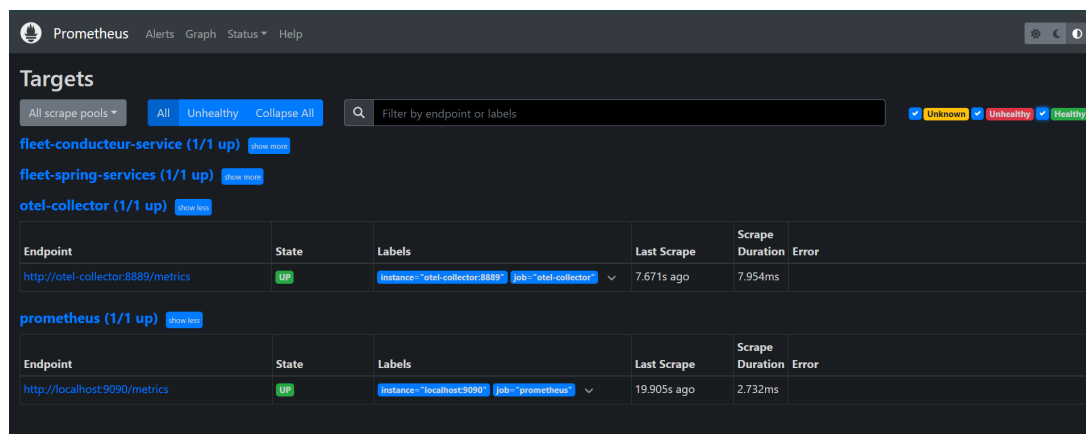


FIGURE 2.3 – Interface Prometheus (Targets Up/Down)

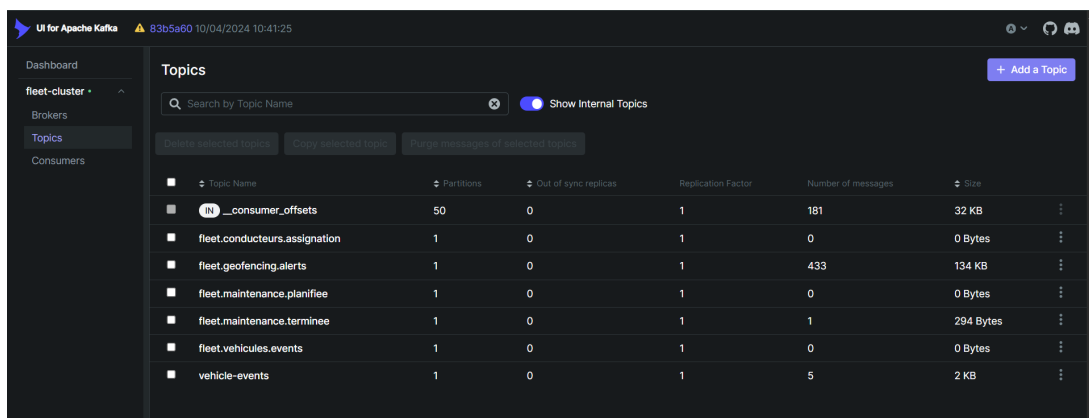
Chapitre 3

Échanges Événementiels et Tests de Charge

3.1 Apache Kafka : Monitoring des Topics

Surveillance des messages échangés de manière asynchrone entre les microservices sur les principaux topics (ex : `fleet.events`). L'utilisation de Kafka garantit :

- Le découplage fort entre le service de géolocalisation et les autres domaines métiers (maintenances, alertes).
- Une haute tolérance aux pannes et une persistance des événements.
- Une absorption efficace (buffering) des pics de sollicitations en cas d'envoi massif de coordonnées GPS.



The screenshot shows the 'UI for Apache Kafka' dashboard. The left sidebar has a menu with 'Dashboard', 'fleet-cluster', 'Brokers', 'Topics', and 'Consumers'. The 'Topics' section is active. The main area shows a table of topics with columns: Topic Name, Partitions, Out of sync replicas, Replication Factor, Number of messages, and Size. The topics listed are: `_consumer_offsets`, `fleet.conducteurs.assignment`, `fleet.geofencing.alerts`, `fleet.maintenance.planifiee`, `fleet.maintenance.terminee`, `fleet.vehicules.events`, and `vehicle-events`.

Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
<code>_consumer_offsets</code>	50	0	1	181	32 KB
<code>fleet.conducteurs.assignment</code>	1	0	1	0	0 Bytes
<code>fleet.geofencing.alerts</code>	1	0	1	433	134 KB
<code>fleet.maintenance.planifiee</code>	1	0	1	0	0 Bytes
<code>fleet.maintenance.terminee</code>	1	0	1	1	294 Bytes
<code>fleet.vehicules.events</code>	1	0	1	0	0 Bytes
<code>vehicle-events</code>	1	0	1	5	2 KB

FIGURE 3.1 – Console UI de Kafka (UI for Apache Kafka ou logs montrant les topics)

3.2 K6 : Script de Charge

Exécution d'un script K6 simulant de multiples utilisateurs concurrents (Virtual Users). Cette étape de Load Testing permet :

- De valider que le temps de réponse de la Gateway reste dans le seuil acceptable (P95 < 500ms).
- De simuler de véritables flux d'utilisation asynchrone (création de véhicules, positionnements).
- D'observer le comportement de l'infrastructure sous contrainte via l'observabilité.

```

X status is 200 (Vehicles)
  0% - ✓ 0 / X 435
X status is 200 (Drivers)
  0% - ✓ 0 / X 435
✓ status is 200 (Gateway OK)

checks.....: 33.33% ✓ 435      X 870
data_received.....: 430 kB 3.8 kB/s
data_sent.....: 113 kB 1.0 kB/s
http_req_blocked.....: avg=20.33µs min=4.6µs med=8.64µs max=2.14ms p(90)=13.07µs p(95)=18.86µs
http_req_connecting.....: avg=7.4µs min=0s med=0s max=1.42ms p(90)=0s p(95)=0s
http_req_duration.....: avg=7.64ms min=923.37µs med=7.29ms max=56.43ms p(90)=13.82ms p(95)=18.3ms
  { expected_response:true }...: avg=2.45ms min=923.37µs med=1.96ms max=31.63ms p(90)=3.54ms p(95)=4.65ms
http_req_failed.....: 66.66% ✓ 870      X 435
http_req_receiving.....: avg=150.81µs min=50.88µs med=128.4µs max=2.45ms p(90)=217.49µs p(95)=268.61µs
http_req_sending.....: avg=59.12µs min=14.31µs med=51.22µs max=2.52ms p(90)=88.6µs p(95)=112.22µs
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=7.43ms min=829.72µs med=7.15ms max=56.15ms p(90)=13.48ms p(95)=17.94ms
http_reqs.....: 1305 11.614026/s
iteration_duration.....: avg=4.02s min=4.01s med=4.02s max=4.07s p(90)=4.03s p(95)=4.04s
iterations.....: 435 3.871342/s
vus.....: 1 min=1 max=20
vus_max.....: 20 min=20 max=20

running (1m52.4s), 00/20 VUs, 435 complete and 0 interrupted iterations
default ✓ [ 100% ] 00/20 VUs 1m50s

```

FIGURE 3.2 – Terminal : Résultat final de l'exécution du script K6 (Summary report)

Chapitre 4

Déploiement Orchestré

4.1 Kubernetes & Helm : Processus de Déploiement

Le déploiement de l'ensemble de la flotte a été automatisé via la création d'un **Chart Helm** personnalisé. Ce chart regroupe tous les manifestes Kubernetes (Deployments, Services, ConfigMaps, Ingress) permettant de déployer l'infrastructure en une seule commande centralisée.

4.1.1 Commandes de déploiement

Voici les étapes exactes et le code exécuté pour déployer notre environnement sur le cluster Kubernetes (Minikube / K3s) :

```
1 # 1. Mise à jour des dépendances du Chart (si besoin)
2 helm dependency update ./fleet-chart
3
4 # 2. Déploiement de l'ensemble de l'architecture dans le namespace
   'fleet'
5 helm upgrade --install fleet-release ./fleet-chart --namespace
   fleet --create-namespace
6
7 # 3. Vérification du statut de la release Helm
8 helm ls -n fleet
9
10 # 4. Suivi du démarrage et de l'état des Pods
11 kubectl get pods -n fleet --watch
```

Listing 4.1 – Installation via Helm et vérification

4.1.2 État des ressources orchestrées

L'orchestration Kubernetes apporte la validation de la santé de nos conteneurs grâce aux sondes `liveness` et `readiness`. Une fois la commande `helm install` exécutée, nous pouvons observer la création structurée des déploiements (ReplicaSets) contrôlant nos pods.

```
C:\Users\leiti\Systeme-de-Gestion-de-Flotte-de-Vehicules>helm list -A
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP
VERSION						
fleet-release-0	default	3	2026-04-20 12:57:32.0431486 +0100 CET	deployed	fleet-management-1.0.0	1.0.
loki-3	fleet-management	1	2026-03-25 11:40:21.2822575 +0100 CET	deployed	loki-stack-2.10.3	v2.9
obs-9.0	fleet-management	1	2026-03-25 11:40:56.2565768 +0100 CET	deployed	kube-prometheus-stack-82.14.0	v0.8
otel-collector-7.0	fleet-management	1	2026-03-25 11:42:11.5168957 +0100 CET	deployed	opentelemetry-collector-0.147.1	0.14
postgres-0	fleet-management	1	2026-03-22 16:05:06.2923568 +0100 CET	deployed	postgresql-18.5.6	18.3
postgres-conducteur-0	fleet-management	1	2026-03-22 16:05:09.4978404 +0100 CET	deployed	postgresql-18.5.6	18.3
postgres-evenement-0	fleet-management	1	2026-03-22 16:05:16.3561403 +0100 CET	deployed	postgresql-18.5.6	18.3
postgres-maintenance-0	fleet-management	1	2026-03-22 16:05:12.7175327 +0100 CET	deployed	postgresql-18.5.6	18.3
redis-1	fleet-management	1	2026-03-22 16:05:22.913711 +0100 CET	deployed	redis-25.3.2	8.6.
strimzi-operator-0	fleet-management	1	2026-03-22 16:23:58.9765287 +0100 CET	deployed	strimzi-kafka-operator-0.51.0	0.51
timescaledb-0	fleet-management	1	2026-03-22 16:05:19.6772351 +0100 CET	deployed	postgresql-18.5.6	18.3
traefik-11	fleet-management	1	2026-03-22 16:02:46.4055546 +0100 CET	deployed	traefik-39.0.6	v3.6

FIGURE 4.1 – Terminal : Sortie de la commande `helm list -A`

fleet-management	alertmanager-obs-kube-prometheus-stack-alertmanager-0	2/2	Running	20 (87m ago)	28d
fleet-management	conducteur-service-964574875-pg4lsq	1/1	Running	11 (87m ago)	31d
fleet-management	conducteur-service-964574875-q428p	1/1	Running	11 (87m ago)	31d
fleet-management	evenement-service-d849446b4-8nr44	1/1	Running	11 (87m ago)	31d
fleet-management	evenement-service-d849446b4-lqg26	1/1	Running	11 (87m ago)	31d
fleet-management	fleet-kafka-dual-role-0	1/1	Running	23 (65m ago)	31d
fleet-management	fleet-kafka-entity-operator-5c6dbc6757-5sph9	2/2	Running	129 (57m ago)	31d
fleet-management	jaeger	1/1	Running	10	28d
fleet-management	localisation-service-7d968b688c-dzk82	1/1	Running	11 (87m ago)	31d
fleet-management	localisation-service-7d968b688c-kkvlw	1/1	Running	11 (87m ago)	31d
fleet-management	loki-0	1/1	Running	10 (87m ago)	28d
fleet-management	loki-promtail-bhsp7	1/1	Running	10	28d
fleet-management	maintenance-service-5f9df8fdb-lzmgh	1/1	Running	11 (87m ago)	31d
fleet-management	maintenance-service-5f9df8fdb-p96nh	1/1	Running	11 (87m ago)	31d
fleet-management	obs-grafana-85bb5fc58d-clbs5	3/3	Running	37 (77m ago)	28d
fleet-management	obs-kube-prometheus-stack-operator-7db67d546-78gfk	1/1	Running	11 (87m ago)	28d
fleet-management	obs-kube-state-metrics-7c9c97f874-9ktsb	1/1	Running	12 (87m ago)	28d
fleet-management	obs-prometheus-node-exporter-hqtdb	1/1	Running	11 (75m ago)	28d
fleet-management	otel-collector-opentelemetry-collector-5c79b856bf-cswq7	1/1	Running	23 (82m ago)	28d
fleet-management	postgres-conducteur-postgresql-0	1/1	Running	15 (77m ago)	31d
fleet-management	postgres-evenement-postgresql-0	1/1	Running	15	31d
fleet-management	postgres-maintenance-postgresql-0	1/1	Running	15 (77m ago)	31d
fleet-management	postgres-postgresql-0	1/1	Running	15 (77m ago)	31d
fleet-management	prometheus-obs-kube-prometheus-stack-prometheus-0	2/2	Running	20 (87m ago)	28d
fleet-management	redis-master-0	1/1	Running	15 (77m ago)	31d
fleet-management	redis-replicas-0	1/1	Running	34 (70m ago)	31d
fleet-management	redis-replicas-1	1/1	Running	34 (65m ago)	31d
fleet-management	redis-replicas-2	1/1	Running	34 (65m ago)	31d
fleet-management	strimzi-cluster-operator-7d9bbbf5d-bpvkt	1/1	Running	25 (76m ago)	31d
fleet-management	timescaledb-postgresql-0	1/1	Running	16	31d
fleet-management	traefik-74f568c58c-kzvxn	1/1	Running	13 (87m ago)	31d
fleet-management	vehicule-service-596548dc5d-6ngmd	1/1	Running	46 (65m ago)	31d
fleet-management	vehicule-service-596548dc5d-lpbbg	1/1	Running	46 (62m ago)	31d

FIGURE 4.2 – Terminal : Sortie de `kubectl get pods -A`